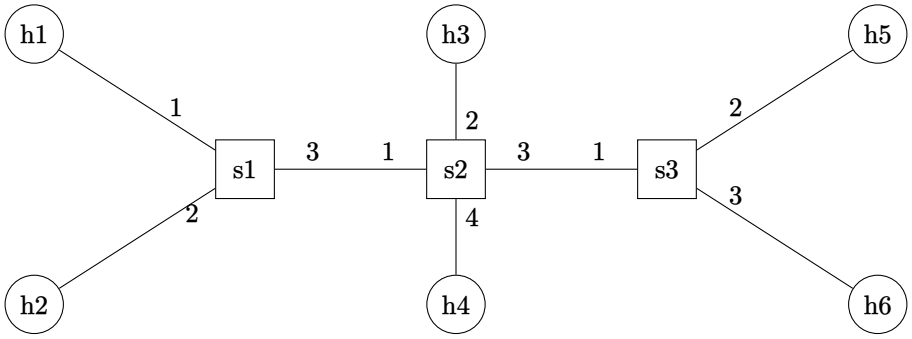
Software Defined Networking – 27 giugno 2025

Esercizio 1

Si consideri la rete in figura. I nodi h1,..., hn contengono tabelle ARP prepopolate per tutta la rete. Gli indirizzi MAC e IP dei nodi h1,..., hn sono MAC1,..., MACn e IP1,..., IPn. Sui nodi h1,..., hn sono in esecuzione processi in ascolto su tutte le porte TCP e UDP di interesse. Il controllore Ryu è collegato ai nodi s1,..., sm. La funzione di gestione dell'evento PacketIn è data nel Programma 1. Le interfacce tra gli switch e il controllore non sono indicate in figura. Se non altrimenti specificato, il TTL iniziale è 200. Negli switch sono preinstallate le regole in Tabella 1.



L'host h1 porta 3322 apre una connessione TCP verso h3 porta 123.

1. Scrivere se $s1$ invia messaggi al controllore. In caso affermativo scrivere quale tipo di messaggio openflow arriva in risposta dal controllore.

   Soluzione: Abbiamo un match con la prima regola, quindi lo switch non invia messaggi al controllore.

2. Scrivere se $s2$ invia messaggi al controllore. In caso affermativo scrivere quale tipo di messaggio openflow arriva in risposta dal controllore.

   Soluzione: Abbiamo un match con la terza regola, quindi lo switch invia un pacchetto al controllore. Il controllore risponde con un messaggio flow-mod e con un messaggio packet-out.

3. Scrivere se $h4$ riceve il pacchetto. In caso affermativo, dire se il pacchetto viene consegnato all'applicazione oppure in quale strato protocollare viene scartato.

   Soluzione: s2 invia il pacchetto in flooding, quindi $h4$ lo riceve. Tuttavia l'indirizzo MAC destinazione è MAC3, pertando il pacchetto viene scartato dalla scheda di rete durante l'esecuzione delle procedure di strato 2.

4. Quale regola il controllore può inserire in s2 per fare in modo che il pacchetto di risposta relativo a questa connessione sia inoltrato all'host corretto senza passare dal controllore?

   Soluzione:

   ```
   priority = 20
   match = [ eth_type=IPv4, ip_dst=IP1, ip_src=IP3, ip_proto=TCP, tcp_dst=3322, tcp_src=123 ]
   action=[ output(1) ]
   ```

Tabella 1: Regole iniziali

Switch s1		
Priority	*Match*	*Action*
5	*	DecTTL, Output(FLOOD)
10	eth_type = IPv4, ip_proto=UDP	Output(CONTROLLER)
15	eth_dst = MAC2	Output(2)
15	eth_dst = MAC1	Output(1)
Switch s2		
Priority	*Match*	*Action*
0	*	DecTTL, Output(FLOOD)
10	eth_type = IPv4, ip_proto = UDP	Output(FLOOD)
10	eth_type = IPv4, ip_proto = TCP	Output(CONTROLLER)
Switch s3		
Priority	*Match*	*Action*
0	*	DecTTL, Output(FLOOD)
15	eth_dst = MAC5	Output(2)
15	eth_dst = MAC6	Output(3)

Programma 1 – Packet In

```
@set_ev_cls(ofp_event.EventOFPPacketIn, MAIN_DISPATCHER)
def packet_in_handler(self, ev):
   msg = ev.msg; datapath = msg.datapath; ofproto = datapath.ofproto;
   parser = datapath.ofproto_parser; in_port = msg.match['in_port']
   pkt = packet.Packet(msg.data); pkt_eth = pkt.get_protocol(eth.eth)
   pkt_ipv4 = pkt.get_protocol(ipv4.ipv4)
   pkt_udp = pkt.get_protocol(udp.udp); pkt_tcp = pkt.get_protocol(tcp.tcp)
   data = msg.data

   if (datapath.id == 2 and pkt_tcp is not None and pkt_ipv4.dst == IP3):
      match = parser.OFPMatch(eth_type=ETH_TYPE_IP,
         ipv4_dst = pkt_ipv4.dst, ip_proto=IPPROTO_TCP,
         tcp_dst=pkt_tcp.dst_port)
      actions = [
         parser.OFPActionOutput(2) ]
      inst = [ parser.OFPInstructionActions(
         ofproto.OFPIT_APPLY_ACTIONS,actions)]
      mod = parser.OFPFlowMod(datapath=datapath, priority=20,
         match=match, instructions=inst)
      datapath.send_msg(mod)

   actions = [parser.OFPActionOutput(ofproto.OFPP_FLOOD)]
   out = parser.OFPPacketOut(datapath=datapath,
            buffer_id=ofproto.OFP_NO_BUFFER,data=msg.data,
            in_port=in_port, actions=actions)
   datapath.send_msg(out)
```

Esercizio 2

Un sistema di sicurezza perimetrale tiene traccia degli accessi non autorizzati a un'infrastruttura di rete tramite un filtro di Bloom. Quando viene rilevato un tentativo di accesso da un IP sospetto, il relativo indirizzo viene inserito nel filtro. Il filtro è costituito da un array di dimensione $m = 2^{16}$ celle e utilizza $k = 4$ funzioni hash indipendenti. Gli IP sospetti osservati finora sono $n = 12\,000$.

5. Calcolare la probabilità che un IP legittimo venga erroneamente identificato come sospetto dal filtro.

> **Soluzione:** Si usa l'usuale formula per la probabilità di falso positivo:
>
> $$p = \left(1 - e^{-\frac{kn}{m}}\right)^k \simeq 0.073$$

6. Calcolare il numero minimo e massimo di celle poste a 1 nel filtro.

> **Soluzione:** In tutto si calcolano kn hash. Il numero minimo si ha quando tutti gli hash collidono, per cui si ottiene $N_{\min} = 1$. Il numero massimo si ha quando gli hash danno luogo a un output diverso, per cui $N_{\max} = \min(m, kn) = 48000$.

Nel caso in cui il filtro dia esito positivo, si effettua un controllo in una cuckoo table per verificare se l'IP è effettivamente sospetto. La cuckoo table è dimensionata in modo ottimo per contenere gi n indirizzi.

7. Calcolare il numero medio di accessi in memoria per verificare se un IP è sospetto, assumendo che il 90% degli indirizzi verificati non siano presenti nel filtro.

> **Soluzione:** Ogni verifica nel filtro di Bloom richiede k accessi. Se il valore è positivo, si effettua anche un controllo nella cuckoo table, che richiede 2 accessi.
>
> $$\text{Accessi medi} = 0.9k + (1 - 0.9)(k + 2) = 0.9 \cdot 4 + 0.1 \cdot (4 + 2) = 3.6 + 0.6 = 4.2$$

8. Qual è lo spazio di memoria necessario per memorizzare il filtro di Bloom e la cuckoo table?

> **Soluzione:** Il filtro di Bloom richiede m bit. La cuckoo table richiede $2n$ celle, ognuna delle quali contiene un indirizzo IP di 4 ottetti, per cui lo spazio è $8n$ ottetti.
>
> In totale, lo spazio richiesto è:
>
> $$S = m + 64n = 2^{16} + 64 \cdot 12000 = 834\,\text{kbit}$$
>
> *In questa soluzione si ipotizza che sia possibile usare un indirizzo IP convenzionale per indicare che la cella è vuota.*

Esercizio 3

Si consideri una rete con 4 router di frontiera identificati dalle lettere A, B, C e D. Il traffico in entrata e in uscita da ogni router è riportato nella seguente tabella. La capacità di ogni collegamento è $V = 100\,\text{Gbit/s}$ e i pacchetti sono tutti di lunghezza $L = 500$ ottetti.

Router	Traffico IN (Gbit/s)	Traffico OUT (Gbit/s)
A	15	20
B	25	35
C	25	20
D	30	20

9. Usando il *metodo gravitazionale*, stimare il traffico tra l'ingresso B e l'uscita C. *Si assuma che il traffico possa entrare e uscire dallo stesso router.*

Soluzione:

$$\text{Traffico} = \frac{B_{IN} \times C_{OUT}}{A_{IN} + A_{OUT} + B_{OUT} + C_{OUT} + D_{OUT}} = \frac{25 \times 20}{95} = 5{,}26\,\text{Gbit/s}$$

Si vuole poi stimare il tipo di traffico entrante da B. Pertanto si campionano i pacchetti con probabilità p e si osserva la rete per $T = 1$ ora. La memoria usata per contare i pacchetti consente di effettuare un incremento ogni $\tau = 10\,\mu\text{s}$.

10. Qual è il valore massimo che è possibile scegliere per p?

Soluzione: Arriva un pacchetto ogni

$$T_p = \frac{L}{V} = \frac{500 \times 8}{100 \times 10^9} = 40\,\text{ns}$$

La massima probabilità di campionamento è quindi

$$p = \frac{T_p}{\tau} = \frac{L}{V\tau} = \frac{40 \times 10^{-9}}{10 \times 10^{-6}} = 4 \times 10^{-3}$$

Si supponga ora che $p = 10^{-4}$.

11. Vengono contati $x = 90\,000$ pacchetti di tipo streaming. Qual è la stima, in Gbit/s, del traffico streaming?

Soluzione:

$$\text{Traffico} = \frac{xL}{Tp} = 1\,\text{Gbit/s}$$

12. Un flusso di $y = 600$ pacchetti di tipo sconosciuto si presenta al router B. Qual è la probabilità che venga campionato almeno un pacchetto?

Soluzione:

$$\Pr\{\text{almeno un pacchetto}\} = 1 - (1-p)^y \simeq 1 - e^{-py} = 1 - e^{-0.06} = 0{,}06$$

Esercizio 4

Una matrice di commutazione crossbar di uno switch 4×4 usa l'algoritmo Take-a-Ticket per lo scheduling delle trasmissioni.

Al tempo $t = 0$ sono presenti i seguenti pacchetti, in ordine dal più recente al più vecchio. Ogni pacchetto è etichettato con il numero dell'interfaccia di uscita cui è diretto. Nel caso in cui si debba operare una scelta casuale, l'algoritmo sceglie sempre in ordine alfabetico. Si assumano pacchetti tutti di lunghezza L.

Ingresso A: 4, 2, 3

Ingresso B: 1, 2, 2

Ingresso C: 4, 2, 3

Ingresso D: 2, 3, 1

13. Quali uscite trasmettono pacchetti durante il primo round?

Soluzione: Uscite 1(D), 2(B) e 3(A).

14. Quali uscite trasmettono pacchetti durante il secondo round?

Soluzione: Uscite 2(A) e 3(C)

Ripetere l'algoritmo assumendo che i pacchetti provenienti dall'ingresso B siano di lunghezza $2L$.

15. Quali uscite trasmettono pacchetti durante il secondo round?

Soluzione: Uscita 3(C)

Si consideri ora uno switch $N \times N$ e uno scenario di traffico uniforme con code sempre sature e pacchetti tutti della stessa lunghezza.

16. Qual è la probabilità che in un generico round tutte le uscite siano occupate? Come cambia questa probabilità al crescere di N?

Soluzione: La probabilità che in un generico round tutte le uscite siano occupate è data da:

$$p = \frac{N!}{N^N} \simeq$$

Al crescere di N, la probabilità tende a zero. Usando l'approssimazione di Stirling abbiamo che:

$$\lim_{N\to\infty} \frac{N!}{N^N} = \frac{\sqrt{2\pi N} N^N}{N^N e^N} = \frac{\sqrt{2\pi N}}{e^N} = 0$$